Day 13 – Deep Learning & Transformers

1. Big Picture

A **neural network** learns patterns by repeatedly adjusting numerical parameters called **weights**.

A **transformer** is a particular neural-network architecture designed to understand relationships between items in a sequence, such as words in a sentence.

Modern systems use transformers for:

- **LLMs:** generating and understanding text
- **Embedding models:** converting text into vectors
- **RAG:** retrieving relevant information and generating grounded answers
- **Multimodal models:** processing text, images, audio, and video

A useful mental model is:

> Neural networks learn patterns. Transformers learn relationships between tokens. LLMs are very large transformers trained on enormous datasets.

---

2. Neural Networks

2.1 What Is a Neural Network?

A neural network is a collection of mathematical operations organized into **layers**.

Each layer receives numbers, transforms them, and passes the result to the next layer.

```
Input → Hidden Layer → Hidden Layer → Output
```

For a text application:

```
Input tokens
    ↓
Token embeddings
    ↓
Multiple neural-network layers
    ↓
Probability of the next token
```

For example, given:

```
The capital of France is
```

the model may output probabilities such as:

```
Paris      0.92
London     0.03
Berlin     0.02
Rome       0.01
Other      0.02
```

The model chooses or samples from these probabilities.

2.2 Layers

A **layer** transforms its input into a more useful representation.

A simplified neural-network layer performs:

[z = Wx + b]

Then it applies an activation function:

[a = activation(z)]

Where:

- (x): input
- (W): learnable weights
- (b): learnable bias
- (z): weighted combination
- (a): output of the layer

Intuition

Imagine each layer as a team of feature detectors.

In an image model:

- Early layers detect edges
- Middle layers detect shapes
- Later layers detect objects

In a language model:

- Early layers may recognize token-level patterns
- Middle layers may capture syntax and relationships
- Later layers may represent meaning and task-specific information

The boundaries are not always this clean, but this is a useful intuition.

2.3 Input, Hidden, and Output Layers

Input layer

Receives the original data.

For text, the raw words are first converted into:

1. Tokens
2. Token IDs
3. Embedding vectors

```
"Transformers are useful"

Tokens:
["Transform", "ers", "are", "useful"]

Token IDs:
[1023, 228, 527, 8491]
```

```
Embeddings:
[[0.12, -0.41, ...],
 [0.38,  0.19, ...],
 ...]
```

Hidden layers

Learn intermediate representations.

A deep neural network may have dozens or hundreds of hidden layers.

Output layer

Produces the final prediction.

Examples:

- Classification probability
- Numeric regression value
- Next-token probability distribution
- Embedding vector

---

3. Activation Functions

Without activation functions, stacking many layers would behave like one large linear transformation.

An **activation function introduces nonlinearity**, allowing the model to learn complex patterns.

3.1 ReLU

[ReLU(x) = max(0,x)]

```
Input:  -3  -1   0   2   5
Output:  0   0   0   2   5
```

ReLU is simple and computationally efficient.

3.2 Sigmoid

Sigmoid converts a value into a number between 0 and 1.

[sigmoid(x) = \frac{1}{1+e^{-x}}]

It is commonly associated with binary classification outputs.

Example:

```
Probability that an email is spam = 0.91
```

Sigmoid is less common in transformer hidden layers because it can suffer from vanishing gradients.

3.3 Softmax

Softmax converts multiple scores into a probability distribution.

Example:

```
Raw scores:
Paris  = 8.2
London = 3.1
Berlin = 2.4

After softmax:
Paris  = 0.99
London = 0.006
Berlin = 0.003
```

In LLMs, softmax is used to generate probabilities over the vocabulary.

3.4 GELU

Transformers commonly use **GELU**, or a related activation such as SwiGLU.

GELU behaves somewhat like a smoother version of ReLU.

Interview-level understanding:

> GELU provides nonlinearity while allowing smoother gradient flow than a hard ReLU cutoff.

---

4. Loss Function

A **loss function** measures how wrong the model is.

Training tries to reduce this loss.

```
Prediction → Compare with correct answer → Calculate loss
```

Example: next-token prediction

Input:

```
The sky is
```

Correct next token:

```
blue
```

Suppose the model predicts:

```
green: 0.60
blue:  0.10
gray:  0.20
other: 0.10
```

Because the correct answer has low probability, the loss will be high.

After training, the model might predict:

```
blue: 0.85
```

The loss becomes lower.

Cross-Entropy Loss

LLMs usually use **cross-entropy loss** for next-token prediction.

Conceptually, cross-entropy penalizes the model when it gives a low probability to the correct token.

The objective is:

> Assign high probability to the correct next token and low probability to incorrect tokens.

---

5. Optimization

Once the loss is calculated, the model must update its weights.

This process involves:

1. Forward pass
2. Loss calculation
3. Backpropagation
4. Weight update

```
Input
  ↓
Forward pass
  ↓
Prediction
  ↓
Loss
  ↓
Backpropagation
  ↓
Update weights
```

5.1 Backpropagation

Backpropagation calculates how much each weight contributed to the error.

It computes a **gradient** for every trainable parameter.

A gradient answers:

> In which direction should this weight move to reduce the loss?

The optimizer then uses these gradients to update the weights.

5.2 Learning Rate

The **learning rate** controls the size of each weight update.

```
New weight = Old weight − Learning rate × Gradient
```

If the learning rate is too large:

- Training may become unstable
- The model may overshoot the best solution

If it is too small:

- Training may be extremely slow
- The model may get stuck or make little progress

5.3 Stochastic Gradient Descent

SGD updates parameters using a small batch of training examples rather than the entire dataset.

Conceptually:

```
Take a batch
→ Calculate predictions
→ Calculate loss
→ Calculate gradients
→ Update weights
→ Repeat
```

Advantages

- Simple
- Memory-efficient
- Can generalize well

Limitations

- May converge slowly
- Sensitive to the learning rate
- Can oscillate in difficult optimization landscapes

5.4 Adam

Adam is an adaptive optimization algorithm.

It adjusts the effective learning rate separately for different parameters.

Adam tracks:

- The recent average gradient direction
- The recent average squared gradient magnitude

Intuition:

> SGD gives every parameter a similar update rule. Adam gives each parameter a more personalized update size.

Adam and variants such as **AdamW** are widely used for training transformers.

Adam vs AdamW

AdamW handles weight decay separately from the gradient update. It is commonly preferred for transformer training because it provides better regularization behavior.

---

6. From Neural Networks to Transformers

Earlier sequence models included:

- Recurrent Neural Networks
- LSTMs
- GRUs

These models process tokens largely one after another.

```
Token 1 → Token 2 → Token 3 → Token 4
```

This sequential processing creates two problems:

1. Training is difficult to parallelize.
2. Information from distant tokens may weaken as it passes through many steps.

Transformers introduced **attention** as the primary mechanism for connecting tokens.

```
Each token can directly examine other relevant tokens.
```

---

7. Transformer Architecture

A transformer is built from repeated transformer blocks.

A simplified transformer block contains:

```
Input representations
        ↓
Self-attention
        ↓
Add and normalization
        ↓
Feed-forward neural network
        ↓
Add and normalization
        ↓
Output representations
```

The two major components are:

1. **Self-attention**
2. **Feed-forward network**

7.1 Self-Attention

Self-attention allows every token to determine which other tokens are relevant to understanding it.

Consider:

```
The animal did not cross the road because it was tired.
```

What does **it** refer to?

Probably:

```
animal
```

To represent the word “it,” the model should pay attention to “animal.”

Self-attention creates direct relationships between these tokens.

---

8. Query, Key, and Value Intuition

Every token produces three vectors:

- **Query**
- **Key**
- **Value**

A useful analogy is a search system.

Query

Represents what the current token is looking for.

> “What information do I need?”

Key

Represents what a token contains or how it should be matched.

> “What kind of information do I offer?”

Value

Represents the actual information passed forward when the token is considered relevant.

> “Here is the information I provide.”

Example

Sentence:

```
The cat sat on the mat because it was tired.
```

When processing **it**:

```
Query from "it":
Which earlier noun am I referring to?

Keys:
"The"     → article
"cat"     → animal/entity
"mat"     → object/location
```

```
"because" → connector

Strongest match:
"cat"

Value from "cat":
Information representing the cat
```

The representation of “it” is then updated using information from “cat” and possibly other relevant tokens.

8.1 Attention Calculation

The attention operation is commonly written as:

[Attention(Q,K,V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V]

Interview interpretation:

1. Compare each query with the keys.
2. Produce relevance scores.
3. Normalize the scores using softmax.
4. Use those scores to combine the value vectors.

You do not normally need to derive the formula unless the interview is mathematically deep.

8.2 Why Divide by (\sqrt{d_k})?

As vector dimensions increase, query-key dot products can become large.

Large values may make softmax overly sharp, producing unstable or weak gradients.

Scaling by (\sqrt{d_k}) keeps the values in a more manageable range.

---

9. Multi-Head Attention

Transformers do not perform only one attention operation.

They use multiple attention heads.

Each head can learn a different relationship.

For example:

```
Head 1 → subject–verb relationships
Head 2 → pronoun references
Head 3 → nearby context
Head 4 → semantic similarity
Head 5 → structural relationships
```

The outputs from the heads are combined.

Conceptually:

> One head gives one perspective. Multi-head attention allows the model to examine the sequence from several perspectives simultaneously.

The exact behavior of individual heads is learned and may not map cleanly to human-defined categories.

---

10. Feed-Forward Network

After attention mixes information across tokens, each token passes through a feed-forward neural network.

```
Attention:
Tokens exchange information.

Feed-forward layer:
Each token transforms the information it now contains.
```

The feed-forward network is generally applied independently to every token position using the same parameters.

It typically contains:

1. Linear projection to a larger dimension
2. Activation function
3. Linear projection back to the model dimension

A large portion of a transformer's parameters may exist in these feed-forward layers.

---

11. Residual Connections and Layer Normalization

Residual connection

A residual connection adds the original input back to a layer's output.

```
Output = Layer(Input) + Input
```

This helps:

- Preserve earlier information
- Improve gradient flow
- Train deep networks more reliably

Layer normalization

Layer normalization keeps activations numerically stable.

It helps prevent values from becoming excessively large or small as they pass through many layers.

Interview-ready explanation:

> Residual connections help information and gradients flow through deep networks, while layer normalization stabilizes training.

---

12. Positional Encoding

Self-attention alone does not inherently understand token order.

Without position information, these sentences would appear to contain the same collection of tokens:

```
Dog bites man.
Man bites dog.
```

But their meanings are very different.

Transformers therefore add positional information to token representations.

```
Final input representation
=
Token embedding
+
Position information
```

12.1 Why Position Matters

Position helps the model understand:

- Word order
- Sentence structure
- Relative distance between tokens
- Which token came before or after another
- Document structure

Example:

```
Position 1: The
Position 2: model
Position 3: generated
Position 4: an
Position 5: answer
```

12.2 Types of Positional Encoding

Sinusoidal positional encoding

The original transformer used fixed sine and cosine functions.

Advantages:

- No additional learned position table
- Can express different frequencies and relative offsets

Learned positional embeddings

The model learns an embedding for each supported position.

Example:

```
Position 1 → learned vector
Position 2 → learned vector
Position 3 → learned vector
```

Rotary Position Embeddings

Many modern LLMs use **RoPE**, or Rotary Position Embeddings.

RoPE incorporates relative-position information by rotating query and key vectors according to token position.

High-level interview explanation:

> Modern positional methods such as RoPE help attention account for the relative distance and ordering between tokens.

---

13. Encoder-Only Models

Examples:

- BERT
- RoBERTa
- DistilBERT

An encoder-only model is designed primarily to understand input text.

Every token can generally attend to tokens on both its left and right.

```
The bank approved the loan.
          ↑
Can inspect both preceding and following context
```

This is called **bidirectional attention**.

Common uses

- Text classification
- Named entity recognition
- Sentiment analysis
- Semantic search
- Reranking
- Embedding generation
- Token classification

Example

Input:

```
The customer wants to close the account.
```

Output:

```
Intent: account_closure
```

Interview summary

> Encoder-only models are strong at understanding and representing complete input sequences.

---

14. Decoder-Only Models

Examples:

- GPT-style models
- Llama
- Mistral
- Qwen
- Claude-style autoregressive models

A decoder-only model generates text one token at a time.

Each token can attend only to previous tokens, not future tokens.

```
The model generated a
                     ↓
Predict next token
```

This is enforced using a **causal attention mask**.

For example, while predicting token 4:

```
Can see:    tokens 1, 2, 3
Cannot see: token 4 or future tokens
```

Common uses

- Chatbots
- Code generation
- Text generation
- Question answering
- Tool-calling agents
- Reasoning tasks
- Document generation

Interview summary

> Decoder-only models are optimized for autoregressive generation, predicting the next token based on previous tokens.

---

15. Encoder-Decoder Models

Examples:

- T5
- BART
- Original Transformer
- Many translation models

These models have two major parts.

```
Input text
    ↓
Encoder
    ↓
Input representation
    ↓
Decoder
    ↓
Generated output
```

The encoder reads the complete input.

The decoder generates the output while attending to:

- Previously generated output tokens
- Encoder representations

Common uses

- Translation
- Summarization
- Text-to-text transformation
- Grammar correction
- Structured generation

Example:

```
Input:
Summarize: Transformers use attention to model token relationships...

Output:
Transformers use attention to understand relationships between tokens.
```

Interview summary

> Encoder-decoder models are useful when an input sequence must be transformed into a different output sequence.

---

16. Model-Type Comparison

Architecture	Main strength	Attention behavior	Typical tasks
Encoder-only	Understanding	Bidirectional	Classification, embeddings, reranking
Decoder-only	Generation	Causal, left-to-right	Chat, completion, coding
Encoder-decoder	Input-to-output transformation	Encoder bidirectional; decoder causal	Translation, summarization

A simple memory technique:

```
Encoder = Understand
Decoder = Generate
Encoder-decoder = Transform
```

---

17. Why Transformers Work Well

17.1 Parallelization

RNNs process sequence steps mostly one by one.

```
RNN:
Token 1 → Token 2 → Token 3 → Token 4
```

Transformers can process many token positions simultaneously during training.

```
Transformer:
Token 1 ─┐
Token 2 ─┼→ Process together
Token 3 ─┼→
Token 4 ─┘
```

This makes transformers much more suitable for training on GPUs and TPUs.

Important nuance:

> Training can process tokens in parallel, but autoregressive decoding still generates tokens sequentially.

17.2 Long-Range Dependencies

In an RNN, information may need to pass through many intermediate steps.

```
Token 1 → Token 2 → Token 3 → ... → Token 100
```

In self-attention, token 100 can directly attend to token 1.

```
Token 100 ──────────────→ Token 1
```

This makes it easier to capture long-distance relationships such as:

- Pronoun references
- Topic consistency
- Relationships across paragraphs
- Code variable usage
- Dependencies in long documents

Transformers are not unlimited, however. They are restricted by their context window and attention implementation.

17.3 Scaling

Transformer performance tends to improve when appropriately increasing:

- Model parameters
- Training data
- Compute
- Context length
- Training quality

This scalability helped enable foundation models and LLMs.

17.4 Flexible Input and Output

Transformers can process many data types after converting them into sequences:

- Text tokens
- Image patches
- Audio frames
- Video frames
- DNA sequences
- Time-series segments

This architectural flexibility supports multimodal models.

---

18. Transformers and LLMs

An LLM is generally a transformer trained on a very large corpus.

For a decoder-only LLM, the primary pretraining task is next-token prediction.

Example:

```
Input:
Machine learning is a field of

Target:
artificial
```

The model repeatedly predicts the next token across billions or trillions of training tokens.

Over time, it learns patterns involving:

- Grammar
- Facts
- Writing styles
- Code structures
- Common reasoning patterns
- Relationships between concepts

Important distinction

An LLM does not retrieve every answer from a traditional database during generation.

It generates output from:

- Learned model parameters
- Current prompt context
- Retrieved context, when RAG is used
- Tool outputs, when tools are used

---

19. Transformers and Embeddings

An **embedding** is a numerical vector representing meaning.

Example:

```
"reset my password"
→ [0.13, −0.42, 0.76, ...]
```

Semantically similar text should have vectors that are close together.

```
"reset my password"
"change my login credentials"
```

Their embeddings should be closer than:

```
"weather in Bangalore"
```

Transformer-based embedding models create these vectors by processing the text and producing a contextual representation.

Embedding pipeline

```
Text
 ↓
Tokenizer
 ↓
Transformer encoder
 ↓
Token representations
 ↓
Pooling
 ↓
Single embedding vector
```

Pooling may use:

- A special classification token
- Mean pooling across token vectors
- A trained pooling mechanism

---

20. Transformers and RAG

RAG combines retrieval with generation.

```
User question
      ↓
Embedding model
      ↓
Query vector
      ↓
Vector database search
      ↓
Relevant document chunks
      ↓
LLM prompt
      ↓
Grounded answer
```

Transformers may appear in several parts of a RAG system.

20.1 Query embedding

A transformer embedding model converts the question into a vector.

```
"How do I reset my corporate password?"
→ Query embedding
```

20.2 Document embedding

The same or a compatible model converts document chunks into vectors.

```
Password reset policy
→ Document embedding
```

20.3 Reranking

A cross-encoder transformer may examine each query-document pair.

```
Query + candidate document
      ↓
Transformer reranker
      ↓
Relevance score
```

A reranker is usually more accurate than simple embedding similarity but more computationally expensive.

20.4 Answer generation

A decoder-only or encoder-decoder transformer receives:

- User question
- Retrieved context
- Instructions

It then generates the final answer.

---

21. RAG Example

Suppose an employee asks:

```
How many days of parental leave do employees receive?
```

Step 1: Embed the question

```
Question → Embedding vector
```

Step 2: Retrieve related chunks

The vector database returns:

```
Chunk 1: Parental leave policy
Chunk 2: Paid time-off policy
Chunk 3: Benefits eligibility
```

Step 3: Rerank

A transformer reranker selects the most directly relevant policy section.

Step 4: Generate

The LLM receives:

```
System instructions
User question
Relevant parental leave policy
```

It generates a grounded answer.

Without RAG, the LLM may rely on outdated or incorrect learned information.

With RAG, the model can answer from current organizational documents.

---

22. Attention Cost and Limitations

Standard self-attention compares every token with every other token.

For (n) tokens, the attention matrix contains roughly:

[n \times n]

Therefore, its time and memory complexity is approximately:

[O(n^2)]

Doubling the sequence length can produce roughly four times as many attention relationships.

This creates challenges for:

- Long documents
- Large context windows
- Inference memory
- Training cost

Approaches used to address this include:

- Sliding-window attention
- Sparse attention
- FlashAttention
- Grouped-query attention
- Retrieval-augmented generation
- Context compression
- State-space and hybrid architectures

For interview purposes:

> Transformers capture long-range relationships well, but standard attention has quadratic cost with respect to sequence length.

---

23. Common Interview Confusions

“Attention is the same as retrieval”

Not exactly.

Self-attention selects and combines information from tokens already inside the model's current context.

Retrieval searches an external corpus or database.

```
Attention:
Search within the current prompt/context.

Retrieval:
Search outside the model in external data.
```

“An embedding model and an LLM are the same”

They may both use transformers, but they are optimized for different outputs.

```
Embedding model:
Text → Vector

Generative LLM:
Text → Next-token probabilities → Generated text
```

“Transformers understand word order automatically”

Self-attention itself does not encode order. Positional information must be introduced.

“Transformers process everything in parallel”

During training, tokens can largely be processed in parallel.

During decoder-only generation, output tokens are generated sequentially.

---

24. Interview Answer Framework

When asked to explain a transformer, use this sequence:

1. **Problem:** Sequence models struggled with parallelism and long-range dependencies.
2. **Core idea:** Self-attention lets every token directly examine relevant tokens.
3. **Mechanism:** Queries match keys, and attention weights combine values.
4. **Order:** Positional encoding supplies sequence-order information.
5. **Block:** Attention is followed by feed-forward layers, residual connections, and normalization.
6. **Architectures:** Encoder-only, decoder-only, and encoder-decoder.
7. **Applications:** LLMs, embeddings, reranking, and RAG.
8. **Trade-off:** Standard attention has quadratic sequence-length complexity.

---

25. Interview Q&A

1. What is the purpose of an activation function?

An activation function introduces nonlinearity. Without it, multiple neural-network layers would collapse into an equivalent linear transformation and could not learn sufficiently complex patterns.

---

2. What is the difference between loss and optimization?

The **loss function** measures how wrong the model's prediction is. The **optimizer** uses gradients to update model parameters so that the loss decreases.

---

3. What is the conceptual difference between SGD and Adam?

SGD updates parameters directly using gradients and a shared learning-rate strategy. Adam maintains moving statistics of gradients and squared gradients, providing adaptive update sizes for different parameters. AdamW is commonly used for transformer training.

---

4. Explain query, key, and value in self-attention.

The query represents what the current token is looking for. Keys represent what other tokens offer for matching. Values contain the information that will be combined. Query-key similarity produces attention weights, which are then applied to values.

---

5. Why do transformers need positional encoding?

Self-attention does not inherently know token order. Positional encoding adds information about where each token appears, allowing the model to distinguish sentences such as “dog bites man” and “man bites dog.”

---

6. What is the difference between BERT and GPT?

BERT is primarily encoder-only and uses bidirectional context, making it strong for understanding tasks such as classification and embeddings. GPT is decoder-only and uses causal attention, making it strong for autoregressive text generation.

---

7. What is causal attention?

Causal attention prevents a token from seeing future tokens. During next-token prediction, each position can attend only to earlier positions, ensuring that the model cannot use information that would not yet be available during generation.

---

8. Why do transformers handle long-range dependencies better than RNNs?

Self-attention creates direct connections between distant tokens. An RNN may need to carry information through many sequential steps, while a transformer token can attend directly to another token far away in the sequence.

---

9. How are transformers used in a RAG system?

Transformer embedding models generate query and document vectors. Transformer rerankers improve candidate ordering. A generative transformer uses the retrieved chunks as context to produce the final grounded answer.

---

10. What is a major limitation of standard self-attention?

Its time and memory requirements grow approximately quadratically with sequence length because every token may interact with every other token. This makes very long contexts computationally expensive.

---

26. Final Memory Map

```
Neural Network
├── Layers
├── Weights and biases
├── Activations
├── Loss
└── Optimization
     ├── SGD
     └── Adam / AdamW

Transformer
├── Token embeddings
├── Positional information
├── Self-attention
│    ├── Query
│    ├── Key
│    └── Value
├── Multi-head attention
├── Feed-forward network
├── Residual connections
└── Layer normalization

Transformer Types
├── Encoder-only → Understand
├── Decoder-only → Generate
```

```
└── Encoder-decoder → Transform

GenAI Usage
├── LLM → Generate text
├── Embedding model → Generate vectors
├── Reranker → Score relevance
└── RAG → Retrieve context and generate answers
```

The most important interview sentence to remember is:

> A transformer uses self-attention to build contextual token representations by dynamically deciding which other tokens matter, while positional information preserves sequence order. This architecture scales effectively for LLM generation, embedding creation, reranking, and RAG.